

程序设计 II 第 13 周 题目汇总（完整版）

来源：Matrix·课程「程序设计 II（25 级 曾坤 计算机学院）」含：完整题面、提交文件、平台提供的初始文件（.hpp/main.cpp 等）。选择题含标准答案。

共 7 项

25 程设II-周 13-课后 1

类型：编程题

题面

物流运输计费系统

题目场景

某物流公司有三种运输方式：陆运（Road）、空运（Air）、海运（Sea）。

需要完成 Road、Air、Sea 三个类以及两个功能：1. 计算所有运输订单的 **总运费**；2. 找出 **单位重量运费最高的运输方式**。

所有运输对象已存放在：`vector<Transport*> transports;`，你只需完成 **两个函数**。

计费规则

基础运费

运输方式

陆运

空运

海运

额外费用：空运若重量 > 100kg，额外收取 **燃油附加费 200 元**

单位重量运费：陆运 2、空运 5、海运 = 总费用 ÷ 重量

输入格式

n

1 名称

2 名称

3 名称

重量

输入示例

```
3
1 卡车 A
2 飞机 B
3 货轮 C
120
```

输出示例

本次总运费: 1180
单位重量运费最高的类型: 空运

提交文件：transport.cpp

平台提供的初始文件

文件 main.cpp：

```
#include <iostream>
#include <vector>
#include "transport.h"
using namespace std;

int main() {
    int n, weight;
    cin >> n;

    vector<Transport*> transports;

    for (int i = 0; i < n; ++i) {
        int type;
        string name;
        cin >> type >> name;

        if (type == 1) transports.push_back(new Road(name));
        else if (type == 2) transports.push_back(new Air(name));
        else transports.push_back(new Sea(name));
    }

    cin >> weight;

    cout << "本次总运费: " << calcTotalCost(transports, weight) << endl;
    cout << "单位重量运费最高的类型: "
         << findHighestCostType(transports, weight) << endl;

    for (auto p : transports) delete p;
    return 0;
}
```

文件 transport.h：

```

#ifndef TRANSPORT_H
#define TRANSPORT_H

#include <string>
#include <vector>

using namespace std;

class Transport {
public:
    string name;
    Transport(string n):name(n){}
    virtual double cost(int weight) const = 0;
    virtual string getType() const = 0;
    virtual ~Transport(){}
};

class Road : public Transport {
public:
    Road(string name);
    double cost(int weight) const override;
    string getType() const override{
        return "陆运";
    }
};

class Air : public Transport {
public:
    Air(string name);
    double cost(int weight) const override;
    string getType() const override;
};

class Sea : public Transport {
public:
    Sea(string name);
    double cost(int weight) const override;
    string getType() const override;
};

double calcTotalCost(const vector<Transport*>& transports, int weight);
string findHighestCostType(const vector<Transport*>& transports, int weight);

#endif

```

25 程设II-周 13-课后 2

类型：编程题

题面

员工奖金计算系统

题目场景

某公司有三种员工：普通员工、经理、实习生。计算所有员工的奖金总额，并找出加班费最高的员工类型。

核心考点：typeid（判断对象实际类型）、dynamic_cast（基类指针转派生类指针，调用派生类特有成员函数）。

每位员工的奖金和加班费已包含在 `vector<ZooWorker*> v`，仅需完善两个函数：

```
double calcTotalBonus(const vector<ZooWorker*>& workers);  
string findHighestOvertimeType(const vector<ZooWorker*>& workers, int hours);
```

计算规则

基础奖金：普通员工 工资 $\times 0.1$ ；经理 工资 $\times 0.2$ ；实习生 工资 $\times 0.05$

额外奖金（需 typeid + dynamic_cast）：经理额外 +500 $\times$ 职级；其他无

加班费：普通员工 小时 $\times 50$ ；经理 小时 $\times (10 + \text{职级} \times 15)$ ；实习生 小时 $\times 30$

输入格式

第一行：员工数量 n ；接下来 n 行（1 姓名 工资 / 2 姓名 工资 职级 / 3 姓名 工资）；最后一行：加班小时数。

输入示例 1

```
3  
1 张三 5000  
2 李四 8000 2  
3 王五 3000  
10
```

输出示例 1

```
本次总奖金: 3250  
本次加班费最高类型: 普通员工
```

用法：`if (typeid(*w) == typeid(Manager)) ; Manager* m = dynamic_cast<Manager*>(w); int level = m->getLevel();`

提交文件：ZooWorker.cpp

平台提供的初始文件

文件 main.cpp：

```

#include "ZooWorker.h"
#include <typeinfo>

int main() {
    vector<ZooWorker*> v;
    int n, type, level, hours;
    string name;
    double salary;

    cin >> n;

    for (int i = 0; i < n; i++) {
        cin >> type >> name >> salary;

        if (type == 1) {
            v.push_back(new ZooWorker(name, salary));
        }
        else if (type == 2) {
            cin >> level;
            v.push_back(new Manager(name, salary, level));
        }
        else if (type == 3) {
            v.push_back(new Intern(name, salary));
        }
    }

    cin >> hours;

    cout << "本次总奖金: " << calcTotalBonus(v) << endl;
    cout << "本次加班费最高类型: " << findHighestOvertimeType(v, hours) << endl;

    for (auto w : v) delete w;
    return 0;
}

```

文件 ZooWorker.h :

```

#ifndef ZOOWORKER_H
#define ZOOWORKER_H

#include <iostream>
#include <string>
#include <vector>
using namespace std;

class ZooWorker {
protected:
    string name;
    double salary;
public:

```

```

ZooWorker(const string& n, double s) : name(n), salary(s) {}
virtual ~ZooWorker() {}
virtual double getBonus() const { return salary * 0.1; }
virtual double getOvertimePay(int hours) const { return hours * 50; }
};

class Manager : public ZooWorker {
    int level; // 经理职级
public:
    Manager(const string& n, double s, int l) : ZooWorker(n, s), level(l) {}
    double getBonus() const override { return salary * 0.2; }
    double getOvertimePay(int hours) const override { return hours *
(10+level*15); }
    int getLevel() const { return level; }
};

class Intern : public ZooWorker {
public:
    Intern(const string& n, double s) : ZooWorker(n, s) {}
    double getBonus() const override { return salary * 0.05; }
    double getOvertimePay(int hours) const override { return hours * 30; }
};

double calcTotalBonus(const vector<ZooWorker*>& workers);
string findHighestOvertimeType(const vector<ZooWorker*>& workers, int hours);

#endif

```

25 程设II-周 13-课后 3

类型：编程题

题面

进程调度模拟系统

三种调度算法：FIFO（选下标最小且未完成）、SJF（选剩余时间最短，同则下标小）、RR（按 0,1,2,3... 轮转，lastIndex 记录，每次最多执行“时间片”秒）。

remaining[i] 为进程 i 剩余执行时间（0 表示完成）；lastIndex 初始 -1。selectProcess 返回下一次执行的进程 ID。周转时间 = 结束时间（到达时间为 0）。

实现四个函数：simulate、FIFOScheduler::selectProcess、SJFScheduler::selectProcess、RRScheduler::selectProcess。

输入格式

第一行：进程数量 n；第二行：n 个整数（所需时间）；最后一行：调度方式（1 FIFO / 2 SJF / 3 RR，RR 时第三行为 3 时间片）。

输入示例 1

```
4
8 4 9 5
1
```

输出示例 1

FIFO 调度顺序：
进程 0 进程 1 进程 2 进程 3
平均周转时间: 16.75

输入实例 2

```
4
8 4 9 5
3 3
```

输出示例 2

RR 调度顺序(时间片=3):
进程 0 进程 0 进程 0 进程 1 进程 1 进程 1 进程 2 进程 2 进程 2 进程 3 进程 3 进程 3 进程 0 进
程 0 进程 0 进程 1 进程 2 进程 2 进程 2 进程 3 进程 3 进程 0 进程 0 进程 2 进程 2 进程 2
平均周转时间: 21.5

已给出的模拟函数框架

```
void simulate(Scheduler* scheduler, const vector<int>& burstTimes) {
    int n = burstTimes.size();
    vector<int> remaining = burstTimes;
    vector<int> completionTime(n, 0);
    vector<int> order; // RR 专用：每秒执行顺序
    vector<int> procOrder; // FIFO/SJF 专用：进程执行顺序
    int currentTime = 0;
    int lastIndex = -1;
    int completed = 0;

    scheduler->printName();

    // 判断是否是 RR 调度器
    bool isRR = (dynamic_cast<RRScheduler*>(scheduler) != nullptr);
    int quantum = 0;
    if (isRR) {
        quantum = dynamic_cast<RRScheduler*>(scheduler)->getQuantum();
    }

    if (!isRR) {
        // FIFO 和 SJF：每个进程只记录一次，连续执行直到完成
        while (completed < n) {
```

```

    int idx = scheduler->selectProcess(remaining, lastIndex);
    if (idx == -1) break;

    lastIndex = idx;
    procOrder.push_back(idx);

    currentTime += remaining[idx];
    completionTime[idx] = currentTime;
    remaining[idx] = 0;
    completed++;
}

for (size_t i = 0; i < procOrder.size(); i++) {
    cout << "进程" << procOrder[i];
    if (i < procOrder.size() - 1) cout << " ";
}
cout << endl;
} else {
    // RR : 每秒记录一次，时间片轮转
    while (completed < n) {
        int idx = scheduler->selectProcess(remaining, lastIndex);
        if (idx == -1) break;

        lastIndex = idx;

        int execTime = min(quantum, remaining[idx]);
        for (int t = 0; t < execTime; t++) {
            order.push_back(idx);
        }
        remaining[idx] -= execTime;
        currentTime += execTime;

        if (remaining[idx] == 0) {
            completionTime[idx] = currentTime;
            completed++;
        }
    }

    for (size_t i = 0; i < order.size(); i++) {
        cout << "进程" << order[i];
        if (i < order.size() - 1) cout << " ";
    }
    cout << endl;
}

// 计算平均周转时间
请完善平均周转时间
}

```

提交文件：Scheduler.cpp

平台提供的初始文件

文件 Scheduler.h :

```
#ifndef SCHEDULER_H
#define SCHEDULER_H

#include <iostream>
#include <string>
#include <vector>
#include <climits>
#include <algorithm>
using namespace std;

class Scheduler {
protected:
    string name;
public:
    Scheduler(const string& n) : name(n) {}
    virtual ~Scheduler() {}
    virtual int selectProcess(const vector<int>& remaining, int& lastIndex) = 0;
    virtual void printName() const { cout << name << "调度顺序:" << endl; }
};

class FIFOScheduler : public Scheduler {
public:
    FIFOScheduler() : Scheduler("FIFO") {}
    int selectProcess(const vector<int>& remaining, int& lastIndex) override;
};

class SJFScheduler : public Scheduler {
public:
    SJFScheduler() : Scheduler("SJF") {}
    int selectProcess(const vector<int>& remaining, int& lastIndex) override;
};

class RRScheduler : public Scheduler {
    int quantum;
public:
    RRScheduler(int q) : Scheduler("RR"), quantum(q) {}
    int selectProcess(const vector<int>& remaining, int& lastIndex) override;
    void printName() const override {
        cout << name << "调度顺序(时间片=" << quantum << "):" << endl;
    }
    int getQuantum() const { return quantum; }
};

void simulate(Scheduler* scheduler, const vector<int>& burstTimes);

#endif
```

文件 main.cpp :

```
#include "Scheduler.h"

int main() {
    int n, type, quantum;
    cin >> n;

    vector<int> burstTimes(n);
    for (int i = 0; i < n; i++) cin >> burstTimes[i];

    cin >> type;

    Scheduler* scheduler = nullptr;
    if (type == 1) scheduler = new FIFOScheduler();
    else if (type == 2) scheduler = new SJFScheduler();
    else if (type == 3) { cin >> quantum; scheduler = new RRScheduler(quantum);
}

    simulate(scheduler, burstTimes);

    delete scheduler;
    return 0;
}
```

25 程设II-周 13-课后 4

类型：编程题

题面

缓存替换策略

设计 CachePolicy 类及派生类，模拟不同缓存替换策略下的页面访问。

基类 CachePolicy：capacity、size、pages[105]、enterTime[105]、recentTime[105]、freq[105]、currentTime、tieRule。构造函数初始化并清空状态，tieRule 默认 1。重载 init(int c) 与 init(int c, int rule)；findPage(int) const（返回下标或 -1）；虚函数 access(int)（命中 1 缺失 0）、name() const、printCache() const。

派生 FIFOCache（淘汰 enterTime 最早，name “FIFO”）、**LRUCache**（命中更新 recentTime，淘汰最久未访问，name “LRU”）、**LFUCache**（命中频率+1 并更新 recentTime，淘汰频率最低；平局按 tieRule：1→enterTime 最小、2→recentTime 最小；name “LFU”）。

要求体现继承、虚函数、运行时多态（CachePolicy*）、函数重载（init）。

输入数据

第一行：capacity 和 m；第二行：m 个页面访问序列；第三行：策略 type（1 FIFO/2 LRU/3 LFU，type=3 时为 3 tieRule）。 $1 \leq \text{capacity} \leq 20$ ， $1 \leq m \leq 100$ 。

输出：第一行 策略名 + Cache Simulation:；第二行 Hits: x Misses: y；第三行 Final Cache: ...

输入: 3 6 / 1 2 3 1 4 5 / 1 → FIFO Cache Simulation: / Hits: 1 Misses: 5 / Final Cache: 4 5 3

输入: ... / 2 → LRU... / Hits: 1 Misses: 5 / Final Cache: 1 4 5

输入: 3 10 / 1 2 1 3 2 4 1 5 2 6 / 3 1 → LFU... / Hits: 4 Misses: 6 / Final Cache: 1 2 6

提交文件：CachePolicy.cpp

平台提供的初始文件

文件 CachePolicy.hpp：

```
#ifndef CACHE_POLICY_HPP
#define CACHE_POLICY_HPP
```

```
class CachePolicy {
protected:
    int capacity;
    int size;
    int pages[105];
    int enterTime[105];
    int recentTime[105];
    int freq[105];
    int currentTime;
    int tieRule;

public:
    CachePolicy();

    void init(int c);
    void init(int c, int rule);

    int findPage(int page) const;

    virtual int access(int page);
    virtual const char* name() const;
    virtual void printCache() const;
};

class FIFOCache : public CachePolicy {
public:
    int access(int page);
    const char* name() const;
```

```
};

class LRUCache : public CachePolicy {
public:
    int access(int page);
    const char* name() const;
};

class LFUCache : public CachePolicy {
public:
    int access(int page);
    const char* name() const;
};

#endif
```

文件 main.cpp :

```
#include <iostream>
#include "CachePolicy.hpp"

using namespace std;

int main() {
    int capacity, m;
    cin >> capacity >> m;

    int accessList[105];
    for (int i = 0; i < m; ++i) {
        cin >> accessList[i];
    }

    int type;
    cin >> type;

    int rule = 1;
    if (type == 3) {
        cin >> rule;
    }

    FIFOCache fifo;
    LRUCache lru;
    LFUCache lfu;

    CachePolicy* policy = nullptr;

    if (type == 1) {
        policy = &fifo;
        policy->init(capacity);
```

```

    } else if (type == 2) {
        policy = &lru;
        policy->init(capacity);
    } else {
        policy = &lfu;
        policy->init(capacity, rule);
    }

    int hit = 0;
    int miss = 0;

    for (int i = 0; i < m; ++i) {
        int result = policy->access(accessList[i]);
        if (result == 1) {
            hit++;
        } else {
            miss++;
        }
    }

    cout << policy->name() << " Cache Simulation:" << endl;
    cout << "Hits: " << hit << " Misses: " << miss << endl;
    policy->printCache();

    return 0;
}

```

25 程设II-周 13-课堂 1

类型：编程题

题面

动物叫声系统

完善 Animal.cpp。菜单选动物（1 狗/2 猫/3 牛）+ 名字（单词不含空格），可连续创建直到输入 0 结束，然后依次输出叫声。基类 virtual void speak() const 输出 "makes a sound."；dog→barks: Woof!，cat→meows: Meow~，cow→moos: Moo~。无效编号输出 "无效选择！" 并继续循环。

输入示例 1

```

1
wangcai
0

```

输出示例 1

wangcai barks: Woof!

输入示例 2

2
cat
1
dog
4
none
0

输出示例 2

无效选择！
cat meows: Meow~
dog barks: Woof!

提交文件：Animal.cpp

平台提供的初始文件

文件 main.cpp：

```
#include "Animal.h"
#include <vector>

using namespace std;

int main() {
    vector<Animal*> animals;
    int choice;
    string name;

    while (true) {
        cin >> choice;

        if (choice == 0) break;
        cin >> name;

        switch (choice) {
            case 1:
                animals.push_back(new Dog(name));
                break;
            case 2:
                animals.push_back(new Cat(name));
                break;
            case 3:
                animals.push_back(new Cow(name));
                break;
            default:
```

```

        std::cout << "无效选择 !" << std::endl;
    }
}

for (auto animal : animals) {
    animal->speak();
}

for (auto animal : animals) {
    delete animal;
}

return 0;
}

```

文件 Animal.h :

```

#ifndef ANIMAL_H
#define ANIMAL_H

#include <iostream>
#include <string>

using namespace std;

class Animal {
protected:
    string name;
public:
    Animal(const string& n);
    virtual void speak() const;
    virtual ~Animal();
};

class Dog : public Animal {
public:
    Dog(const string& n);
    void speak() const override;
};

class Cat : public Animal {
public:
    Cat(const string& n);
    void speak() const override;
};

class Cow : public Animal {
public:
    Cow(const string& n);
    void speak() const override;
};

```

```
};
```

```
#endif
```

25 程设II-周 13-课堂 2

类型：编程题

题面

程序员计算器

完善派生类 ProgrammerCalculator。功能：1 两整数相加；2 数组求和（基类版）；3 数组平均值（派生版，整数除法向下取整）；4 转二进制（n=0 返回“0”，无前导 0）；5 表达式 a&b 按位与（输出“a&b=结果”换行，格式错输出“表达式错误”返回 0）；0 退出输出操作次数。每次 compute 时 callCount++。

输入示例 1

```
1
10 20
2
3
1 2 3
3
3
10 20 30
4
42
5
10&6
0
```

输出示例 1

```
30
数组求和
6
数组平均值
20
101010
10&6=2
2
操作次数: 3
```

输入示例 2

```
5
10+20
4
```


0
0

输出示例 2

表达式错误

0
0

操作次数: 0

注意：成员变量是否需要初始化

提交文件：Calculator.cpp

平台提供的初始文件

文件 main.cpp：

```
#include "Calculator.h"
```

```
int main() {
    ProgrammerCalculator calc;
    int choice;

    while (true) {
        cin >> choice;
        if (choice == 0) break;

        switch (choice) {
            case 1: {
                int a, b;
                cin >> a >> b;
                cout << calc.compute(a, b) << endl;
                break;
            }
            case 2: {
                int n;
                cin >> n;
                int* arr = new int[n];
                for (int i = 0; i < n; i++) cin >> arr[i];
                cout << calc.Calculator::compute(arr, n) << endl;
                delete[] arr;
                break;
            }
            case 3: {
                int n;
                cin >> n;
                int* arr = new int[n];
                for (int i = 0; i < n; i++) cin >> arr[i];
                cout << calc.compute(arr, n) << endl;
                delete[] arr;
                break;
            }
        }
    }
}
```

```

    }
    case 4: {
        int n;
        cin >> n;
        cout << calc.toBinary(n) << endl;
        break;
    }
    case 5: {
        string expr;
        cin >> expr;
        cout << calc.evaluate(expr) << endl;
        break;
    }
}
}

cout << "操作次数: " << ProgrammerCalculator::getCallCount();
return 0;
}

```

文件 Calculator.h :

```

#ifndef CALCULATOR_H
#define CALCULATOR_H

#include <iostream>
#include <string>
using namespace std;

class Calculator {
protected:
    static int callCount;

public:
    Calculator() {}
    virtual ~Calculator() {}

    int compute(int a, int b) {
        callCount++;
        return a + b;
    }

    int compute(const int arr[], int size) {
        callCount++;
        int sum = 0;
        for (int i = 0; i < size; i++) sum += arr[i];
        cout << "数组求和" << endl;
        return sum;
    }
}

```

```

virtual int evaluate(const string& expr) {
    return 0;
}

static int getCallCount() {
    return callCount;
}
};

class ProgrammerCalculator : public Calculator {
public:
    using Calculator::compute;

    int compute(const int arr[], int size); // 计算数组平均值
    string toBinary(int n);                // 转二进制
    int evaluate(const string& expr) override; // 表达式计算
};

#endif

```

25 程设II-周 13-理论题

类型：选择题（20 题）

第 1 题

在 C++ 中，面向对象程序设计中的“多态”主要指的是：

- A. 一个类只能生成一种固定类型的对象
- B. 同一个操作或接口在不同对象上可以表现出不同的行为
- C. 一个函数只能被调用一次
- D. 所有类都必须拥有完全相同的数据成员

答案：B

第 2 题

下列关于 C++ 多态的说法，正确的是：

- A. 函数重载属于运行时多态
- B. 虚函数机制主要用于实现运行时多态
- C. 多态只能通过模板实现
- D. 只要两个函数名称相同，就一定发生运行时多态

答案：B

第 3 题

在 C++ 中，将基类成员函数声明为 virtual 的主要意义是：

- A. 使该函数只能在基类对象中调用
- B. 使该函数在编译时一定绑定到基类版本
- C. 允许通过基类指针或引用调用时，根据实际对象类型选择函数版本
- D. 使派生类不能重新定义该函数

答案：C

第 4 题

下列哪种情况最能体现 C++ 的运行时多态？

- A. 同一个类中定义多个同名但参数不同的函数
- B. 通过基类指针调用虚函数，实际执行派生类中重写的版本
- C. 使用 int、double 等不同类型变量进行算术运算
- D. 在类中定义私有成员变量，限制外部访问

答案：B

第 5 题

关于函数重载与运行时多态的关系，下列说法正确的是：

- A. 函数重载依靠虚函数表实现
- B. 函数重载通常在编译阶段确定调用哪个函数
- C. 函数重载必须发生在基类和派生类之间
- D. 函数重载要求函数的返回值类型必须不同

答案：B

第 6 题

在 C++ 中，若希望通过基类指针调用函数时能够执行派生类中重写的版本，基类中的该函数通常应声明为：

- A. static
- B. virtual
- C. const
- D. friend

答案：B

第 7 题

下列关于虚函数的说法，正确的是：

- A. 虚函数只能在派生类中声明，不能在基类中声明

- B. 虚函数的调用一定在编译时确定
- C. 虚函数可以使基类指针或引用调用函数时发生动态绑定
- D. 虚函数必须是静态成员函数

答案：C

第 8 题

下列代码的输出结果是：

```
class Base {
public:
    virtual void show() { cout << "Base"; }
};
class Derived : public Base {
public:
    void show() { cout << "Derived"; }
};
int main() {
    Derived d;
    Base* p = &d;
    p->show();
    return 0;
}
```

- A. Base
- B. Derived
- C. BaseDerived
- D. 编译错误

答案：B

第 9 题

下列代码的输出结果是（show 非 virtual）：

```
class Base {
public:
    void show() { cout << "Base"; }
};
class Derived : public Base {
public:
    void show() { cout << "Derived"; }
};
int main() {
    Derived d;
    Base* p = &d;
    p->show();
    return 0;
}
```

- A. Base
- B. Derived
- C. BaseDerived
- D. 编译错误

答案：A

第 10 题

使用基类指针或引用调用虚函数时，实际调用哪个函数版本主要取决于：

- A. 指针或引用变量的名称
- B. 指针或引用的静态类型
- C. 指针或引用所绑定对象的实际类型
- D. 函数在源文件中的书写顺序

答案：C

第 11 题

下列关于函数重载的说法，正确的是：

- A. 函数重载要求函数名不同，但参数列表相同
- B. 函数重载要求函数名相同，但参数列表不同
- C. 函数重载只能发生在基类和派生类之间
- D. 函数重载必须依靠 virtual 关键字实现

答案：B

第 12 题

下列哪一组函数声明可以构成函数重载？

- A. void fun(int x); 和 int fun(int x);
- B. void fun(int x); 和 void fun(double x);
- C. void fun(int x); 和 void fun(int y);
- D. int fun(); 和 int fun();

答案：B

第 13 题

关于函数重载和函数覆盖，下列说法正确的是：

- A. 函数重载通常发生在同一作用域中，函数覆盖通常发生在基类和派生类之间
- B. 函数重载必须使用 virtual 关键字，函数覆盖不需要使用 virtual 关键字
- C. 函数覆盖要求函数名相同即可，参数列表可以不同
- D. 函数重载属于运行时多态，函数覆盖属于编译时多态

答案：A

第 14 题

如果派生类中定义了一个与基类虚函数同名，但参数列表不同的成员函数，下列说法正确的是：

- A. 该函数一定正确覆盖了基类虚函数
- B. 该函数不会覆盖基类中参数列表不同的虚函数
- C. 该函数会使基类虚函数自动变成非虚函数
- D. 该函数一定会导致程序运行时错误

答案：B

第 15 题

下列代码的输出结果是（引用 + virtual print）：

```
class Base { public: virtual void print() { cout << "Base"; } };
class Derived : public Base { public: void print() { cout << "Derived"; } };
int main() {
    Derived d;
    Base& r = d;
    r.print();
    return 0;
}
```

- A. Base
- B. Derived
- C. BaseDerived
- D. 编译错误

答案：B

第 16 题

下列代码的输出结果是（Base b = d; + virtual show）：

```
class Base { public: virtual void show() { cout << "Base"; } };
class Derived : public Base { public: void show() { cout << "Derived"; } };
int main() {
    Derived d;
    Base b = d;
    b.show();
    return 0;
}
```

- A. Base
- B. Derived
- C. BaseDerived

- D. 编译错误

答案：A

第 17 题

Base b = d; 体现的现象是：

- A. 动态绑定
- B. 对象切片
- C. 函数重载
- D. 向下转型

答案：B

第 18 题

下列关于静态类型和动态类型的说法，正确的是：

- A. 静态类型由变量声明决定，动态类型由对象实际所属类型决定
- B. 静态类型只能在程序运行时确定
- C. 动态类型一定与静态类型完全相同
- D. 只有普通对象才有动态类型，指针和引用没有动态类型相关问题

答案：A

第 19 题

下列代码的输出结果是（Derived::f(int) 隐藏基类虚函数 f()，Base* p 调 f()）：

```
class Base { public: virtual void f() { cout << "Base"; } };
class Derived : public Base { public: void f(int x) { cout << "Derived"; } };
int main() {
    Derived d;
    Base* p = &d;
    p->f();
    return 0;
}
```

- A. Base
- B. Derived
- C. BaseDerived
- D. 编译错误

答案：A

第 20 题

关于 C++ 多态、虚函数和函数重载，下列说法正确的是：

- A. 只要派生类中出现与基类同名的函数，就一定发生运行时多态

- B. 函数重载主要根据返回值类型区分不同函数
- C. 通过基类指针或引用调用虚函数时，可能根据对象的实际类型决定调用版本
- D. 非虚函数也一定会根据对象的动态类型进行动态绑定

答案：c
